

Ciclo 1

Santiago Murillo Ibáñez

Escuela Colombiana de Ingeniería Julio Garavito

Desarrollo Orientado a Objetos

Luis Andrés Gómez

8 de septiembre de 2026

1. ¿Cuáles fueron los mini-ciclos definidos? Justifiquenlos.

Mini-Ciclo 1 Estructura del Proyecto

Estado: Completo

Objetivo: Entender el flujo de como funciona una maquina tragamonedas y estructurar su construcción.

Lista:

- Crear el proyecto en BlueJ.
- Mirar que clases del paquete shapes pueden ser de utilidad en el desarrollo del proyecto.
- Mirar que clases necesitan extenderse o hacerse más pequeñas.
- Definir las clases del proyecto SlotMachine
- Definir las responsabilidades que tiene cada clase
- Realizar el diagrama de clases con sus atributos y posibles métodos auxiliares
- Realizar por cada método de la guía su diagrama de secuencia
- Definir los siguientes Mini-Ciclos

Mini-Ciclo 2 Estructura de SlotMachine

Estado: Completo

Objetivo: A través de los diagramas de secuencia definir la estructura de validación y llamada de métodos en Wheel o Symbols.

Lista:

- Definir los atributos de la clase SlotMachine la cual se va a relacionar con Wheel y Symbol (Genera una arrayList por cada uno).
- Definir isVisible y ok como booleanos para definir las reglas de negocio
- Implementar SlotMachine(): Su creador
- Implementar makeVisible (): Hace visible la interfaz gráfica.
- Implementar makeInvisible (): Oculta la interfaz gráfica del simulador
- Implementar addWheel(int index): Añade una nueva rueda a la máquina en la posición especificada.
- Implementa delWheel(int index): Elimina una rueda existente de la máquina en la posición especificada.
- Implementa addSymbol(String color, String shape): Añade un nuevo símbolo (definido por color y forma) a la colección disponible.
- Implementa delSymbol(String color, String shape): Elimina un símbolo específico de la colección disponible.
- Implementar placeSymbol(int wheelPos, String color): Establece el símbolo seleccionado en la rueda indicada
- Implementar spin(int wheelPos): Rueda una rueda específica
- Implementar spin(): Rueda todas las ruedas de la máquina

- Implementar `String[] symbols()`: Retorna los símbolos del catálogo
- Implementar `String[] configuration()`: Retorna los colores de los símbolos visibles actualmente
- Implementar `distinctSymbols()`: Retorna el número de distintos colores visibles
- Implementar `isJackpot()`: Evalúa si todas las ruedas muestran el mismo símbolo
- Implementar `exit()`: Termina la simulación limpiando las listas
- Implementar `ok()`: Indica el estado de la última acción realizada
- Implementar `addMultipleWheels(int num)`: Agrega tantas ruedas al final de mi maquina
- Implementar `delMultipleWheels(int num)`: Quita tantas ruedas al final de mi maquina
- Implementar `wheelAt(int pos)`: Obtiene la rueda en la posición especificada
- Implementar `findSymbol(String color)`: Busca un símbolo en la lista por su color.

Mini-Ciclo 3 Crear y administrar ruedas

Estado: Completo

Objetivo: A través de los diagramas de secuencia definir como `SlotMachine` llama métodos en `Wheel`, además de definir sus atributos y relación con `Symbols`

Lista:

- Definir `SimboloActual`, `isVisible`
- Implementar `Wheel(ArrayList<Symbol> symbols)`: Constructor de la rueda ya instaciada con un simbolo aleatorio
- Implementar `spin(ArrayList<Symbol> symbols)`: Hace girar la rueda y selecciona un símbolo aleatorio de la lista.
- Implementar `setSymbol(Symbol symbol)`: Asigna un símbolo específico a la rueda.
- Implementar `getColor()`: Retorna el color del símbolo actual o null si no tiene.
- Implementar `getActualSymbol()`: Retorna el objeto `Symbol` que está asignado actualmente a la rueda.
- Implementar `makeVisible()`: Muestra la rueda visualmente.
- Implementar `makeInvisible()`: Oculta la rueda visualmente.

Mini-Ciclo 4 Crear y administrar Symbols

Estado: Completo

Objetivo: A través de los diagramas de secuencia definir como `SlotMachine` llama métodos en `Symbols`, además de definir sus atributos y relación con el paquete `Shapes`.

Lista:

- Definir color, forma, isVisible y las tres figuras de shapes como atributos.
- Implementar Symbol (String color, String forma): Constructor para instanciar un nuevo Symbol.
- Implementar getColor(): Obtiene el color identificador del símbolo.
- Implementar getForm(): Obtiene el tipo de forma geométrica del símbolo.
- Implementar makeInvisible(): Oculta la figura del símbolo de la pantalla.
- Implementar makeVisible(): Muestra la figura del símbolo de la pantalla.
- Implementar moveHorizontal(int distance): Desplaza horizontalmente la figura visual para alinearla en su columna.
- Implementar moveVertical(int distance): Desplaza verticalmente la figura visual en la pantalla.
- Implementar createFigure(String color, String forma): Crea la figura correspondiente según el valor de 'forma' y le aplica el color.
- Implementar ColorCSS(String colorCSS): Normaliza la cadena para asegurar compatibilidad con nombres de colores estándar CSS y convierte la cadena a minúsculas y elimina espacios extras.
- Implementar createFigure(String color, String forma): Crea la figura correspondiente según el valor de 'forma' y le aplica el color.

Mini-Ciclo 5 RectangleBorder

Estado: Completo

Objetivo: Esta es la parte visual que se encarga de encerrar los símbolos actuales de las ruedas y ser el marco general de la maquina tragamonedas. A partir de cuatro rectángulos de ancho 0.

Lista:

- Definir los atributos que son cuatro rectángulos, isVisible, width, height y thickness.
- Implementar RectangleBorder(int width, int height, int thickness, String color): Constructor para un rectángulo de solo bordes.
- Implementar adjustRectangle(): Ajusta las dimensiones y posiciones de las 4 líneas para formar el borde.
- Implementar changeSize(int newWidth, int newHeight): Cambia el tamaño del marco de bordes.
- Implementar changeColor(String color): Cambia el color de los bordes.
- Implementar makeVisible(): Muestra el marco en la pantalla
- Implementar makeInvisible(): Oculta el marco de la pantalla.
- Implementar moveHorizontal(int distance): Desplaza el marco horizontalmente.
- Implementar moveVertical(int distance): Desplaza el marco verticalmente.

Mini-Ciclo 6 Pruebas de escritorio sin interfaz

Estado: Completo

Objetivo: Mirar que la acciones de SlotMachine son correctas usando pruebas de escritorio

Lista:

- Por cada método de Slot debe reflejarse en los Arraylist de SlotMachine.
- Verificar el método ok () el cual nos indica que el flujo para acciones correcta se realiza.
- Configurar restricciones que fallen o mejoras de código.

Mini-Ciclo 6 Implementar Interfaz grafica

Estado: Completo

Objetivo: Mostrar la lógica de la maquina tragamonedas a lo visual

Lista:

- En la clase de Slot se implementa el atributo de RectangleBorder y el metodo reorderVisualWheels(): El cual Reorganiza las posiciones X de cada rueda para mantener una separación consistente
- En la clase Wheel implementar una posiciónX y RectangleBorder border en cada una de las ruedas existentes.
- En la clase Wheel implementar moveA(int nuevaPosicionX) el cual reposiciona dinámicamente la rueda y su figura gráfica a una nueva coordenada X.

Mini-Ciclo 7 Definir estado ganador

Estado: Completo

Objetivo: Definir un estado ganador al realizar una acción y llegar a un Jackpot

Lista:

- En la clase SlotMachine definir showMessageTemporal(String mensaje, int milisegundos): El cual me indica si se realizo un Jackpot
- En la clase Canvas implementar los métodos necesarios para mostrar el mensaje en lienzo.

Mini-Ciclo 8 Probar interfaz gráfica como lógica combinada.

Estado: Completo

Objetivo: Al haber completado el ciclo realizar varias pruebas para mirar que su ejecución haya sido correcta

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿por qué?

En términos se encuentra completado, porque se realizaron todos los mini-ciclos además de que se testearon de una manera casera. Además, el Astah contiene los diagramas de secuencia y de clase que refleja el código de BlueJ en su totalidad. Por otro lado el simulador cumple con los estándares brindados en la guía tanto como lógica y funcional.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

Santiago Murillo 25 horas

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

Diseñar, implementar y probar desde cero el proyecto el cual se realizó de manera individual durante una amena y media, debido a esto se demuestra que con ayuda de los diagramas y planeación se pudo completar de manera satisfactoria.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

La parte del Canvas y la interfaz gráfica. Esta se soluciono leyendo el documento de Oracle y artículos de dibujo en BlueJ. Además esta se hizo acompañada con IA implementaría que mostraba como se debe estructurar la clase Canvas y cómo funciona cada parte de esta.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

7. Considerando las prácticas XP incluidas en los laboratorios. ¿cuál fue la más útil? ¿por qué?

Planeación, ya que gracias a ella se pudo completar el proyecto de manera satisfactoria

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

- Escuela Colombiana de Ingeniería. (2026). Desarrollo Orientado Por Objetos [DOPO-POOB] Proyecto Inicial Ciclo No 1. Documento de requisitos y diagrama UML. De esta se baso el proyecto y fue la mas importante
- ICPC. (2025). Problem I: Slot Machine. Maratón de Programación Internacional.
- [All Packages \(Java SE 17 & JDK 17\)](#)